13. Софтуерно инженерство и неговото място като дял от знанието. Софтуерен процес и модели на софтуерни процеси. Концепция за многократна употреба.

1. Софтуерното инженерство – какво е софтуер, видове софтуер, същност и обхват на софтуерното инженерство.

Софтуер – представлява:

- Инструкции (компютърни програми), които при изпълнение осигуряват желаните характеристики, функциониране и производителност
- Структури от данни, които дават възможност на програмите адекватно да обработват информация
- Документи, които описват работатата и използването на програмите

Софтуер(2) -Компютърни програми и асоцииранатa с тях документация.

Други специфични характеристики на софтуера са:

- Той е логическа едиица
- Процесът на разработка е специфичен и различен от тази на материални обекти
- Той се разработва, а не произвежда
- Не се амортизира
- Сложен е и има тенденция да става все по-сложен и труден
- Абстрактност
- Надеждност
- Преодбладаващата част от софтуера продължава да е “custom build”

Софтуерен продукт – софтуер, предназначен да се продава на клиент/потребител. Разграничават се два типа софтуерни продукти:

- Общи (generic) – самостоятелни продукти/системи, които се продават на пазара и са предназначени за всеки потребител, който иска да ги купи
- Поръчани (customized) – продукти/системи, които са възложени от отделен клиент и се разработват специално за него.

Разграничават се следните категории софтуер:

- Системен – съвкупност от програми, предназначени да обслужват други програми. Има за цел да улесни работата на хардуера. Примери са ОС, драйвери за устройства, компилатори и пр. Характеризира се с:
    - Интензивна комуикация с хардуера
    - Интензивно използване от много потребители
    - Съвместно използване на ресурси
    - Сложно управление на процеси
    - Сложни структури от данни
    - Многобройни външни интерфейси
- Приложен – състои се от самостоятелни програми, които решават специфична нужда на бизнеса. Приложенията обработват бизнес или технически данни, чрез което улесняват бизнес операциите или вземането на решения.
- Научен – обслужва нуждите на отдените дялове от фундаменталните науки. Приложенията са свързани основно със симулации на системи, автоматизирано проектиране...
- Вграден – намира се в дадена система и се използва да реализира и управлява определени функции вместо крайния потребител или системата.
- Продуктова линия – съвкупност от софтуерни системи, които притежават общи черти, задоволяващи нуждите на определен търговски сегмент или предопределена функция.

Разработени са от общи основни компоненти, което намалява цената на разработката на цялата гама от продукти. Специфично е, че всички продукти от линията се издават едновременно.

- Уеб приложения – в най-простата си форма са множество от свързани файлове хипертекст. Съвременните уеб приложения представляват сложни среди, осигуряващи изчислителни функции и съдържание на крайния потребител и се интегрират с бази данни и други приложения.
- Изкуствен интелект – използват се нечислови алгоритми за решаване на сложни проблеми, които не подлежат на изчисления или директен анализ.
- Open source – софтуер, който позволява свободен достъп и използване на първичния му код, както и условията за разпространението му удовлетворяват определена лицензия, позволяваща потребителите да изучават, променят и подобряват софтуера.
- Legacy software – съществуващ софтуер, който е разработен преди няколко десетилетия и продължава да се използва, като непрекъснато е бил променян, за да задоволи промени в бизнес изискванията на платформите. Ключовите характеристики на наследения софтуер са – дълъг живот и критична важност за бизнеса.

Софтуерно инженерство

Дефиниция 1
Прилагането на систематичен, дисциплиниран, количествено измерим поход към разработването, функционирането и поддръжката на софтуера, както и изледване на подходите в тези дейности. Софтуерното иженерство се касае до теориите, методите и tool-овете за професионална разработка на софтуер.

Дефиниция 2
Софтуерното инженерство е инженерна дисциплина, касаеща се до всички аспекти на продукцията на софтуер. Софтуерните инженери трябва да прилагат систематичен и организиран подход към работата си и да използват подходящи инструменти и техники, зависещи от решавания проблем, ограниченията върху разработката и наличините ресурси.

Дефиниция 3
Софтуерното инженерство е дисциплина, която интегрира процеси, методи и средства за разработване на софтуер.

Обхват на софтуерно инженерство – софт. инж. обхваща процеси, методи, софтуерни модели на процеси, качество.
Обхвата на софтуерното инженерство е изключително обширен. Най-общо включва 5 аспекта:

- Исторически – как се е появила дисциплината
- Икономически – цели се разработката на най-икономически ефективното решение
- Поддръжка на готовите решения
- Изисквания, анализ, дизайн – подходи за намаляване на грешките при разработка
- Екипна разработка – техники за управление и организиране на екипите.

2. Софтуерен процес – фази и основни дейности, видове модели и езици за моделиране, шаблони за описание.

Софтуерен процес:

- Последователност от предсказуеми стъпки
- Рамка за задачите, които да се изпълнят, за да се изгради висококачествен софтуер
- Установяващ базата за управление на софтуерния проект и контекста, в който се прилагат техническите методи, създават се работните продукти, осигурява се качеството
- Описание на разработката на софтуер, която включва някои или всички от изброените дейности, организирани по такъв начин, че накрая се създава работещ качествен софтуер.

- Множество дейности, чиято цел е разработката или еволюцията на софтуер.

Етапи при разработване на софтуер

- Анализиране и дефиниране на изскванията
- Проектиране на системата
- Проектиране на програмата
- Писане/кодиране на програмата
- Unit testing
- Integration testing
- Тестване на системата
- Доставяне на системата
- Поддръжка

Основни дейности в софтуерен процес

- ***Комуникация – цели събиране и разбиране на изискванията към софтуера***
- ***Планиране – създава се план за бъдещата работа по разработката на софтуера като се описват рисковете, необходимите ресурси, работните продукти, които ще се произведат, и времеви график на работата***
- ***Моделиране – включва събиране и анализ на изискванията и проектиране на системата.***
- ***Конструиране – включва генериране на код и тестване.***
- ***Внедряване – софтуерът се предоставя на клиента. Клиентът оценява продукта.***

Модел на софтуерен процес – опростено представяне на софтуерен процес, представен от определена перспектива.

Видове модели

- В зависимост от гледната точка:
    - *Workflow model* **–** описва последователноттта от дейности, които се извършват в процеса на разработка. Отделните дейности се описват входните и изходните им параметри и зависимостите една от друга.
    - *Dataflow model* – описва множеството от дейности, които се извършват от гледна точка на преобразуване на данни.
    - *Role/Action model* – описва отелните роли на хората, които участват в процеса на разработане на софтуер, и дейностите, които всяка от тези роли трябва да извършва.
- Описателни или предписателни
    - *Описателни* – описват историята на това как е протекло разработването на софтуерна система
    - *Предписателни* – предписват как би трябвало да се разработи нова софтуерна система. Използват като насоки и рамки за организиране и структуриране на това как трябва да се извършват дейностите, свързани с разработването на софтуер и в какъв ред.

Езици за моделиране на процеси **–** Езиците и формализмите за моделиране на процес се използват за представяне по прецизен и изчерпателен начин на следните характеристики на софтуерния процес:

- Дейностите, които трябва да се извършат, за да се постигнат целите на процеса;
- Ролите на хората, участващи в процеса;
- Структурата и същността на артефактите, които се създават и поддържат;
- Средствата, които се използват.

Видове езици за моделиранре на процеси:

- по езикови класове
    - *дескриптивни* – използват правила или тригери, базирани са на bottom-up метода като отделят специално внимание на времето заразработка и изключенията, които могат да възникнат.
    - *мрежово базирани* – представят процесите на мрежи Петри. Протичането на процеса е лесно разбираемо, но могат да се появят проблеми с абстракцията.
    - *Императивни* – базирани на езиците за програмиране. Процесът е представен като програма.
    - *Хубридни* –кобинират елементи от няколко парадигми в един език.
- спрямо организацията на езика
    - *Entity-Relation* – организирани са като същности, с връзки между тях
    - *Role-Interaction* – фокусират севъху ролите и техните взаимоотношения. Ролите се изпълняват от организационните елементи, а взаимоотношенията са различните взаимовръзки между тях.
    - *Object-Oriented* – вкл. аспектите на процеса в организационния модел. Основната единица е обект, който комбинира данни и функцоналност в един пакет.
- спрямо формалността
    - *Формални* – представя се чрез формален синтаксис и семантика, има ясно дефинирани правила за употребата им и смисъл на конструкциите им.
    - *Полуформални* – обикновено имат графично представяне с формален синтаксис, но не и формална семантика.
    - *Неформални* **–** няма строго дефинирани правила, а смисълът на конструкциите идва от употребата им.

Шаблони за описание на процес – шаблон, който описва доказан, успешен подход и/или последователност от действия за разработване на софтуер. Представлява структурирано описание на процес, което е е метод за описание на важните характеристики на софтуерния процес. Описват какво трябва да се направи. Според нивото си на абстракция са:

- *Task process patterns* – описва отделни стъпки за извършване на действие или задача, която е част от процеса.
- *Stage process patterns* – дефинира стъпките, които се извършват итеративно в рамките на една базова дайност за процеса.
- *Phase process patterns* – дефинира последователнстта и взаимодействието между базови дейности, които се случват в рамките на процеса.

Всеки шаблон трябва да съдържа:

- *Начален контекст*
- *Проблем*
- *Решение*
- *Краен (резултатен) контекст*
- *Свързани шаблони*
- *Начини на употреба*

3. Сравнителен анализ на описателни модели на софтуерен процес

Сравнение съгласно основните характеристики

Моделът на водопада и на бързата разработка представляват постъпкови модели на разработка. Моделът на бързата разработка е високоскоростна адаптация на този на водопада. При тях се последователно се изпълняват основните дейности при разработката на софтуер (събиране и анализ на изисквания, планиране, проектиране, конструиране и внедряване). При модела на водопада всяка дейност трябва да е завършена и работните продукти да бъдат одобрени преди започването на следващата фаза и всяка фаза завършва със създаването на определени документи. Друга важна

характеристика е ясно разграничения процес. При модела на бързата разработка се определят изискванията, планира се изпълнението на проекта и функционалността се разделя на отделни модули, които всеки от екипите моделира и конструира. Разработката завършва с интеграцията на разработените модули и внедряването на готовия продукт.

Спираловидния модел е итеративен/цикличен поход на разработка, който често се представя като спирала. Всяка итерация по спиралата представлява стъпка от разработката. В началото на всяко завъртане по спиралата се извършва анализ на риска. Възможно е за една и съща дейност да се направят няколко последователни завъртания – например за анализ на изискванията. Може да има множество milestones.

Фазовите модели целят доставянето на системата на части. Разликата между постъпковия (инкрементален) модел и итеративния е, че първият доставя само част от цялата функционалност на всяка стъпка, докато итеративния доставя цялостната софтуерна система в началото, макар и част от функционалността да е в примитивна форма. При постъпковия на всяка следваща стъпка реализира нова функционалност, като използва модела на водопада за тези цели. При итеративния модел всяка итерация доставя все по-завършена версия на софтуера – не се добавя нова функционалност, а само се усъвършенства съществуващата. Много често се прилага комбинация между постъпков и итеративен процес – на всяка стъпка се добавя или подобрява функционалност.

При прототипния модел се цели изграждането на прототип за изясняване на изискванията (thrown-away) или бърза доставка на системата (еволюционен).

Методът на формалната трансформация включва трансформиране на спецификацията на системните изисквания до изпълнима програма. При трансформирането трябва да се запази коректността и ясно да се покаже, че изпълнимата програма съответства на спецификацията

Сравнение съгласно проектите, за които са подходящи

Моделът на водопада и спираловидния процес са подходящи за големи проекти. Първият се прилага при проекти с ясни изисквания, ясно дефинирани роли, повтаряеми проекти и такива, в които времето и бюджетът не са критични. Спираловидният процес също е подходящ при разработка на големи системи, но характерното е, че се фокусира върху оценката на риска при всяка итерация. Той обикновено е комбинация между прототипния модел и модела на водопада.

За малки проекти, при които се цели кратка разработка, е подходящ методът на бързата разработка. Единственото изискване е функционалността да може да се раздели на модули, които да се разработят в рамките на 60-90 дни и да са налични достатъчно човешки ресурси, които да работят по отделните модули.

При проекти с неясни изисквания са подходящи спираловидния и прототипния модел. Обикновено прототипния модел с разработването на thrown-away прототип се използва за постигането на яснота относно изискванията.

При липса на досттъчно човешки ресурси обикновено се прилага постъпковия процес.

При наличие на технологични рискове обикновено се прилага постъпковия процес.

При необходимост за бързо доставяне на системата се използват постъпковия, итеративния модел, еволюционния прототип или модела на бързата разработка.

Прототипния модел може да се използва самостоятелно или в комбинация с други модели.

Методът на формалната трансформация се прилага при критични системи, за които трябва да са гарантирани сигурността и отказоустойчивостта на софтуера, преди да започне използването му.

Сравнение по недостатъци

Основните проблеми при модела на водопада са, че е трудно приложим в реалните проекти, тъй като те рядко следват последователния поток на разработка. Също така обикновено е трудно всички изисквания да бъдат формулирани в началото на проекта и е трудно да се реагира на променящи се изисквания. Основната спънка е трудността на въвеждане на промени след започване на процеса. Друг недостатък е, че клиентът няма достъп до междинни версии или отделни модули на софтуера.

Основният проблем при прототипния модел е, че прототипът може да използва значителни ресурси и да не отговори на очакванията.

Проблемите при модела на бързата разработка са, че изисква значителни човешки ресурси при проекти за големи приложения. Може да се прилага единствено при софтуер, който може да бъде подходящо разделен на отделни модули. Многото интерфейси между модулите влияят негативно върху производителността. Моделът не е подходящ при използването на непознати и нови технологии, т.к. е твърде рисков в тези случаи.

При спираловидния модел е трудно да се убедят клиентите, че процесът на разработка е контролируем, а не безкраен цикъл. Неоткриването на основен риск може да доведе до неуспех. Задължително е участието на разработчици с компетентност за оценка на рисковете.

Недостатъците при постъпковия и итеративния модел са, че клиента трябва да участва активно в проекта. Също така моделът може да доведе до scope creep (постепено разширяване на обхвата).

Методът на формалната трансформация изисква специфични умения от разработчиците, трудно е да се дефинират поведенческите аспекти на системата. Прилагането му е скъп и бавен процес.

4. Концепцията за многократна употреба.

Съществуват два концептуални подхода за многокртна употреба на софтуер:

- Базиран на създаване на програмни единици:
    - Шаблони на проектирането
    - Програмна генерация
- Базиран на съществуващи програмни единици:
    - COTS продукти
    - Компонентно-базирано разработване
    - Аспектно ориентирано разработване
    - Използване на съществуващи системи
    - Application frameworks

Предимства на многократна употреба:

- Увеличена надеждност
- Съвместимост със стандарти
- Намалено време за разработка
- Увеличена продуктивност
- Подобряват комуникацията с външни софтуерни системи (software system interoperability)
- Разработка от по-малко хора
- Намалява цените на разработка и поддръжка
- По-малък риск
- Ефективно използване на специалисти

Проблеми на многкратната употреба:

- Увеличени цени за поддръжка
- Липса на поддръжка на tool-ове
- Трудно е да се намерят, разберат и адаптират компоненти за преизползване

Многократна упротреба, базирана на системи

Реализира се чрез:

- Използване на цяло приложение, което да се конфигурира, така че да е съвместимо с околната среда
- Интегриране на 2 или повече системи в нова

За целите на многократна употреба, базирана на преизползване на готови системи се изпозват 2 подхода:

- интеграция на COTS продукти
- разработка на базата на поточни линии

Многократна употреба, базирана на COTS

COTS системите обикновено са цели приложения, предлагащи API. Те са при приложими в широк кръг области като са най-разпространени в системите за електронна търговия.

При разработката с COTS продукти е възможно да съществуват повече от един потенциални кандидати за интегриране в разрботваната система. Обикновено всеки такъв продукт има собствен формати и структура на данните и е небходимо да бъде разработен протокол за обмен на данни между модулите. Също така най-често COTS продуктите предлагат повече функционалност, отколкто е необходима за

съответната интеграция. В този случай е необходимо да бъде предодвратен достъпа до ненужнта функционалност и разработката на wrapper модул.

Основното предимство на разработката чрез интеграция на COTS продукти е по-бързата разработка и евентуало по-ниската цена. Проблемите при този подход са:

- Липса на контрол върху функционалността и производителността
- Понякога интеграцията на COTS продукти е доста труден процес
- Липса на контрол върху развитието и появата на нови версии на продуктите
- Недостатъчна поддръжка от производителите на COTS продуктите

Поточна линия (product line)

Поточните линии представляват приложения с обобщена функционалност, които могат да се адаптират и след това да се използват в специфична област. Адаптацията предполага някои от следните подходи:

- Конфигурация на компонент или подсистема
- Добавяне на нови компоненти в системата
- Избор на компоненти от предварително подготвени библиотеки
- Промяна на подсистеми

Компонентно-базирана разработка

Дефиниция на компонент

Градивна единица на софтуерните системи, вътрешната структура и процесите, на която са видими само за интегратора на системата, но не и за останалите компоненти. Компонентите могат да осъществяват връзка помежду си единствено чрез входно-изходни точки, наречени интерфейси.

Компонетите предоставят услуги, без значение от платформата на която се изпълняват или програмния език, на който са реализирани. Те са независими самостоятелно изпълними единици. Основната цел при компонентите е те да не се компилират преди интеграция с други компоненти.

Компонентите са по-общо онятие от обектите. Всеки компонент може да е изграден от 1 или повече обекти. Всеки компонент може да се изпълнява самостоятелно и чрез тях се постига по-високо ниво на абстракция.

Компонентите предоставят следните интерфейси:

- Входен (requires) интерфейс - Дефинира какво трябва да се предостави на компонента, така че изпълнението му да стане според спецификацията
- Изходен (provides) интерфейс - Дефинира услугата (или резултата от нея), която компонентът предоставя, в следствие на изпълнението му и при условие, че са удовлетворени изискванията на входния интерфейс

Компонентни модели

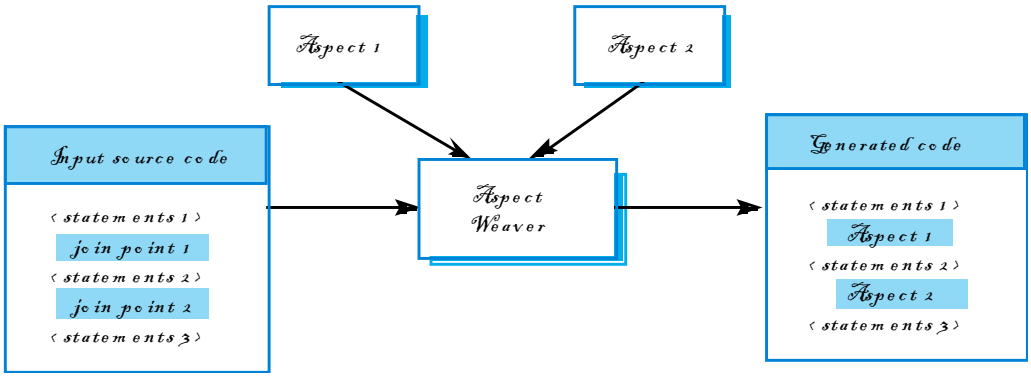
Представляват дефиниция на стандарт за реализация, документиране и метод за разпространение на компоненти. Те определят и правилата за дефиниция на интерфейсите. Примери за компонентни модели са EJB, COM+, .NET.

Аспектно-ориентирано разработване

Аспектно-ориентираното разработване засяга т.нар. разделяне на функциите (*separation of concerns*). Често е много трудно те да се отделят чрез стандартните методи на проектиране. Например:

- Всички компоненти трябва да наблюдават състоянието си
- Всички компоненти трябва да изпълняват политики по сигурността

Припокриващата се функционалност се разработва като аспекти, които динамично се вмъкват в програмата.



Application frameworks

Frameworks са дизайни на подсистеми, съставени от колекция от абстрактни и конкретни класове и интерфейсите между тях. Подсистемите се имплементират чрез добавянето на компоненти и инстанцирането на абстрактните класове във framework-а.

Шаблони на проектирането

Шаблонът на проектирането (*design pattern*) е техника за многократна употреба на някакво абстрактно знание за дадена задача и решението й. Те включват описание на задачата и най-съществените моменти от решението. Абстрактността предполага възможност за прилагане в различни ситуации. Най-често използваните свойства на класовете са наследяване и полиморфизъм.

Класификация на шаблоните на проектирането:

- Според предназначението
    - Съзидателни (*creational*) – за създаване на обектите
    - Структурни *(structural)* – за улесняване на интеграцията на класове и обекти
    - Поведенчески (*behavioral*) – представят методи за комуникация между обектите
- Според обхвата
    - Шаблони на класовете - засягат отношения между класовете, които не се променят след компилацията

- Шаблони на обектите - засягат отношения между обекти. Тези отношения може да се променят по време на изпълнение

Програмна генерация

Програмните генератори използват вече дефинирани шаблони и алгоритми. Те са вградени в генератора и може да се параметризират. Програмата се генерира автоматично.

Програмната генерация е възможна, когато може да се дефинира съответствие между понятията от приложната област и програмния код. Използва се специфичен (за проблемната област) език

Програмната генерация е ефективен метод за разработка, приложим в ограничен набор от приложни области.

13. Софтуерно инженерство и неговото място като дял от знанието. Софтуерен процес и модели на софтуерни процеси. Концепция за многократна употреба.

1. Софтуерното инженерство – какво е софтуер, видове софтуер, същност и обхват на софтуерното инженерство.

Софтуерът се използва от всеки един човек в обществото. Софтуерът се проектира и изгражда от софтуерни инженери. Софтуерните инженери разглеждат софтуера като съставен от програми, документи и данни, необходими за проектирането и изграждането на системата

Софтуерът представлява (IEEE)

- **Инструкции (компютърни програми)** които при изпълнение осигуряват желаните характеристики, функциони–рane и производителност;
- **Структури от данни,** които дават възможност на програмите адекватно да манипулират информация.
- **Документи,** които описват работата и използването на програмите.

Софтуерът, който е предназначен да се продава на клиент/ потребител, се нарича продукт. Могат да се разграничат два типа софтуерни продукти общи (generic) и поръчани (customized/ бутикови) продукти. Общите таргетират голяма група от клиенти и потребители, докато бутиковите се правят за конкретен клиент и са съобразени изключително с неговите нужди и изисквания.

Независимо дали софтуерът е бутиков или с общо предназначение, могат да се идентифицират няколко категории софтуер:

- **Системният софтуер** е съвкупност от програми, които обслужват други програми. Системният софтуер се характеризира с интензивна комуникация с хардуера. Пример за системен софтуер са операционните системи, драйверите, компилаторите, асемблери.
- **Приложният софтуер** се състои от самостоятелни програми, които решават специфична нужда на бизнеса или потребителите. Може да се раздели на няколко големи групи комуникационни приложения (мейл клиенти, уеб браузъри, скайп), мултимедийни приложения (плейъри, графични програми, игри), аналитичен софтуер (статистически програми), collaboration софтуер (уики, блогове, трак), enterprise софтуер (ERP, CRM, BPM) и др.
- **Научен софтуер** обслужват нуждите на конкретна наука . Ползват се в научно–изследвателски проекти, за симулации, проектиране, анализ и др.

- **Вграден (εμβεδδεδ) софтуер** нарича се вграден защото се вгражда в различни от класическия компютър хардуерни системи телефони , фотоапарати , автомобили , перални , микровълнови и т.н. Може да е насочен както към потребителски, така и към бизнес нужди.
- **Продуктова линия** съвкупност от приложения, които притежават общи черти или се допълват в задоволяването на конкретна категория потребителски нужди. Продуктова линия е ΜΣ Οφφιχε пакета. Ключово при продуктовите линии е, че има преизползване на софтуерен компонент, на документ или процес при разработката на всяко приложение от пакета. Това преизползване е предварително дефинирано и планирано.
- **Уеб приложения** – множество от свързани файлове хипертекст, които представят информация чрез текст и прости графики. Съвременните уеб приложения обикновено са многослойни и комуникират интензивно помежду си посредством мрежовата среда. Уеб приложенията могат да бъдат силно натоварени в пиков момент. За уеб приложенията е важна естетиката и сигурността на данните.
- **Изкуствен интелект** този софтуер ползва нечислови алгоритми за решаване на сложни проблеми, които не подлежат на директен анализ. Примери за ИИ са някои приложения в роботиката, приложения за разпознаване на образи и звук, умни търсачки.
- **Отворен код (οπεν σουρχε)** всяка от изброените категории може да е с отворен код. Отвореният код и свободният софтуер са нова философия в света на ИТ. Свободният софтуер дава право на потребителите да го ползват, да преглеждат и редактират кода, да разпространяват програмата. Приложенията с отворен код дават същите права, но със следните ограничения:
    - Първичният код трябва да бъде включен, когато се разпространява софтуерът и неговият автор да е видим в явен вид;
    - Повторното разпространение на приложението трябва да става по силата на същата лицензия и правила за разпространение на отворен код.
- **Наследен софтуер** – Съществуващ софтуер, който е разработен преди няколко десетилетия и продължава да се използва, като непрекъснато е бил променян, за да задоволи промени в бизнес изискванията и платформите (хардуерни и софтуерни). Наследеният софтуер се характеризира с
    - Дълъг живот
    - Критична важност за бизнеса
    - *Лошо качество*

Софтуерното инженерство е инженерна дисциплина, която се занимава с всички аспекти на проектирането, разработването и

преизползването на висококачествен софтуер. Като инженерна дисциплина то поставя в центъра си един проблем, анализира го, декомпозира го на подпроблеми, систематизира подпроблемите и търси тяхните решения, които да интегрира в едно крайно решение.

Софтуерното инженерство е прилагането на систематичен, дисциплиниран, количествено измерим подход към разработването, функционирането и поддръжката на софтуера. Софтуерното инженерство трябва да се базира на общо съгласие в рамките на организацията по отношение качеството.

2. Софтуерен процес – фази и основни дейности, видове модели и езици за моделиране, шаблони за описание.

Софтуерният процес е *последователност от предвидими стъпки* – план, който следвате за да създадете продукт или система в срок и с високо качество.
Софтуерният процес може да се разглежда и като *рамка за задачите*, които даден екип от софтуерни инженери трябва да изпълни, за да реализира висококачествен софтуер.
Обикновено дейностите, които трябва да се извършват в рамките на софтуерния процес, са:

- Анализ и дизайн на изискванията;
- Проектиране на системата;
- Проектиране на програмата;
- Писане на код;
- Тестване на единиците;
- Интегрирано тестване;
- Системно тестване;
- Доставяне и внедряване на системата;
- Поддръжка;

Така споменатите дейности дефинират и съответните роли – аналитик, проектант, програмист, тестер, инструктор и др.
Основните цели на софтуерния процес са:

- Ефикасност;
- Възможност за поддръжка (maintainability);
- Предвидимост/ предсказуемост (predictability);
- Повторяемост (repeatability);
- Качество;
- Постоянно усъвършенстване;
- Проследяване на прогреса;

Моделът на софтуерен процес е опростено описание на софтуерен процес, което е представено от определена перспектива. В зависимост от перспективата могат да се разграничат следните модели:

- Workflow model – последователността от действия, които се извършват, описани с техните входно-изходни параметри;
- Dataflow model – проследява преобразуването на данни и при какви дейности става това;
- Role/action model – описва ролите в екипа и съответните дейности, за които всяка роля отговаря;

Целта на моделите на софтуерния процес е да го направят разбираем за всички участници в него и да служи на ръководителите на проекта да проследяват прогреса и евентуално да намират несъответствия и пропуски в процеса.

Езиците за моделиране на процес се използват за детайлното и изчерпателно представяне на *дейностите, ролите на хората, структурата и същността на артефактите и средствата*.
Има три основни парадигми езици за моделиране на процеси:

- **Дескриптивни** – ползват bottom-up метода за описание, като обръщат внимание предимно на времето за разработка и изключенията, които може да настъпят.
- **Мрежово базирани** – представят процесите като мрежи (най-често мрежи на Петри).
- **Императивни** – базирани на езиците за програмиране, представят процеса като програма.

Съвременните езици за моделиране представлятат комбинация от тези три парадигми.

Тъй като има много езици за моделиране на процеси при избора на такъв за даден проект трябва да се водим от следните цели:

- Разбираемост на модела на процеса;
- Лесен дизайн и поддръжка на модела на процеса;
- Симулация и оптимизация на процеса;

Шаблонът е описание на общ проблем, но в допълнение дава идея за неговото решение. Тази идея се ползва милиони пъти, но решението винаги е уникално, защото шаблонът описва **какво** трябва да се направи, а не **как** трябва да се направи.

Шаблонът на процес е шаблон, който описва доказан, успешен подход и/или последователност от действия за разработване на софтуер.
Антишаблон на процес описва последователност от действия за разработване на софтуер, които са доказано неефективни.

3. Сравнителен анализ на описателни модели на софтуерен процес – модел на водопада, прототипен модел, модел на бързата разработка, спираловиден модел и др.

Моделите на софтуерен процес биват два типа – описателни и предписателните. Първи описват как е протекъл или протича софтуерния процес, докато другите описват как би трябвало да протича софтуерния процес. Предписателните модели на процеси дефинират специално множество от дейности, задачи, milestones, и работни продукти, които са необходими за създаването на софтуер с високо качество.

Ще разгледаме най-разпространените предписателни модели на софтуерни процеси.

3.1. **Waterfall**

- най-старият структуриран модел за разработка на софтуер

- систематизиран и последователен подход към разработването на софтуер, който включва *събиране на изисквания, планиране, анализ и проектиране, имплементация, тестване, внедряване и поддръжка*;
- лесен за разбиране;
- тромав и твърде много бюрократщина, трудно се реагира на промени;
- приложим само в проекти, където изискванията са абсолютно ясни;

3.2. **Модел на бързата разработка (RAD model)**

- Основната цел е кратък цикъл на разработката;
- "Високоскоростна" адаптация на модела на водопада чрез паралелна разработка на системата, което се реализира с помощта на обособяване на модули;
- Скъп модел, поради увеличената консумация на човешки ресурси;
- Компонентно базирани архитектури с цел преизползване на стари компоненти;
- Ако се ползват нови технологии, много вероятно да няма готови компоненти;

3.3. **Еволюционни модели**

3.3.1. **Инкрементален модел**

- Системата не се доставя като едно цяло, а вместо това процесът на разработка и доставянето са разделени на стъпки, като всяка стъпка доставя само част от цялата функционалност.
- На идентифицираните потребителски изисквания се присвояват приоритети и тези с по-висок приоритет се реализират в първите стъпки.
- След като започне разработката на една стъпка, изискванията не се променят.
- Комбинира елементи на модела на водопада, но приложени на отделни стъпки

3.3.2. **Итеративен модел**

- В самото начало доставя цялостната софтуерна система, макар и част от функционалността да е в примитивна форма
- При всяка следваща итерация не се добавя нова функционалност, а само се усъвършенства съществуващата

3.3.3. **Приемущества и недостатъци**

- Необходимостта от активно участие на клиентите по време на изпълнение на проекта може да доведе до закъснения.
- Уменията за комуникация и координация са от особено голямо значение при разработката и ако не са на достатъчно добро ниво, водят до проблеми.
- Неформалните заявки за подобрения след завършването на всяка стъпка могат да доведат до объркване.
- Този модел може да доведе до т. нар. "*scope creep*" – бавно и постепенно разширяване на обхвата на приложението, без процесът да е сходящ
- Прилага се, когато организацията няма достатъчно човешки ресурс за цялостната реализация в определен срок - в разработването на по-ранните версии участват по-малко хора и в зависимост от обратната връзка, получена от клиентите, могат да се присъединят още разработчици на следващите итерации
- Когато с итерациите може да се управляват технологичните рискове - итерация, която изисква използването на нова технология или продукт

може да се планира по-късно с цел да има достатъчно време да се усвои или да се достави новият продукт

3.4. **Прототипен модел**

- Създаване на основните потребителски интерфейси, без да има някакво значително кодиране.
- Разработване на съкратена версия на системата, която изпълнява ограничено подмножество от функции.
- Използване на съществуваща система или компоненти от система, за да се демонстрират някои функции, които ще бъдат включени в разработваната система

3.4.1. **Еволюционен прототип**

- Цел - да достави работеща система на крайния потребител

3.4.2. **Изхвърлен (throw-away) прототип**

- Цел - да подпомогне специфицирането на изискванията към софтуера

3.4.3. **Приемущества и недостатъци**

- Прототипният модел може да използва значителни ресурси, а като резултат прототипът да не успее да удовлетвори очакванията.
- Прототипът може да доведе до лошо проектирана система, ако самият той стане част от крайния продукт.
- Прототипният модел не е подходящ за използване при разработване на софтуерни системи, където проблемът е добре разбран и интерфейсът е ясен и прост
- Прилага се в проекти, където не са достатъчно ясни потребителските изисквания и дизайнът на софтуерната система
- Прилага се както самостоятелно, така и в комбинация с други модели на процеси - модел на водопада, спираловиден модел, постъпков модел и т.н.

3.5. **Спираловиден модел**

- Спираловидният модел е еволюционен модел на софтуерен процес, който съчетава прототипния модел и модела на водопада
- Движещият фактор е анализ на риска
- Основни характеристики: итеративен/цикличен подход и наличие на множество от точки на прогреса (anchor point milestones)
- Може да се окаже трудно да се убедят клиентите, че процесът на разработка е контролируем, а не е безкраен цикъл.
- Изисква се участието на разработчици с компетентност за оценка на рисковете.
- Ако не се идентифицира и открие някой основен риск, това може да доведе до неуспех.
- Прилага се при разработване на големи (large-scale) софтуерни системи.

3.6. Избор на подходящ модел на процес

3.6.1. Изборът зависи основно от два фактора:

- Организационната среда (променяща се, неопределена, стабилна)
- Същността на приложението
- Компетентност и опит на разработчиците
- Използвани технологии
- Бюджет

4. Концепцията за многократна употреба.

В софтуерното инженерство често се търсят решения на сходни проблеми. За целта няма нужда „топлата вода да се открива" отново. Добрите софтуерни инженери постоянно мислят да създават продукт, част от който или целия да могат да преизползват и в друг проект. Това води до повишаване качеството на продукта и намаляване цената му.

4.1. Многократна употреба, базирана на съществуващи програмни единици

4.1.1. Интеграция на COTS (Commercial-Off-The-Shelf) продукти

- COTS системите обикновено са цели приложения, които предлагат API (Application Programming Interface)
- Приложимо в широк кръг области
- Широко разпространение намира в системите за електронна търговия (e-shop, e-mail systems и т.н.)

4.1.1.1. Основни въпроси

- Кои COTS продукти предлагат най-подходяща функционалност?

Възможно е да съществуват повече от един потенциални кандидати за интегриране в системата

- Как ще се обменят данни между модулите?

Всеки модул има собствен формат и структура на данните

- Кои част от продукта ще се използва

Най-често се предоставя повече функционалност, от това което е нужно и трябва да се предотврати достъпа до ненужната функционалност.

4.1.1.2. Проблеми

- Липса на контрол върху функционалността и производителността
- Понякога интеграцията на COTS продукти е доста труден процес
- Липса на контрол върху развитието и появата на нови версии на продуктите
- Недостатъчна поддръжка от производителите на COTS продуктите

4.1.2. Компонентно-базирано софтуерно инженерство

Модел на процес, който стъпва изключително на принципа на преизползването е моделът на компонентно базираното разработване на софтуер. Този модел е комбинация от еволюционния и спираловидния модел.

4.1.2.1. Компонент

- Компонентните са по-общо понятие от обектите
- Всеки компонент може да е изграден от един или няколко обекта
- Може да се изпълняват самостоятелно
- Чрез компонентите се постига по-високо ниво на абстракция
- Предоставят услуги, без значение от платформата на която се изпълняват или програмния език, на който са реализирани
- Независими самостоятелно изпълними единици
- Целта е компонентите да не се компилират преди интеграция с други компоненти

Градивна единица на софтуерните системи, вътрешната структура и процесите, на която са видими само за интегратора на системата, но

не и за останалите компоненти. Компонентите могат да осъществяват връзка помежду си единствено чрез входно-изходни точки, наречени интерфейси

- *Примери за компонентни модели*
  - *EJB (Enterprise Java Beans)*
  - *COM+, .NET*
  - *CCM (Corba Component Model)*

4.1.3. Аспектно-ориентирано разработване

4.1.3.1. **Същност**

Аспектно-ориентираното разработване засяга т.нар. разделяне на функциите (*separation of concerns*).

Често е много трудно те да се отделят чрез стандартните методи на проектиране. Например:

- Всички компоненти трябва да наблюдават състоянието си
- Всички компоненти трябва да изпълняват политики по сигурността

Припокриващата се функционалност се разработва като аспекти, които динамично се вмъкват в програмата

4.2. Многократна употреба, базирана на създаване на програмни единици

4.2.1. Шаблони на проектирането

4.2.1.1. **Същност**

- Шаблонът на проектирането (*design pattern*) е техника за многократна употреба на някакво абстрактно знание за дадена задача и решението й
- Описание на задачата и най-съществените моменти от решението
- Абстрактността предполага възможност за прилагане в различни ситуации
- Най-често използваните свойства на класовете са наследяване и полиморфизъм

4.2.1.2. **Защо се използват шаблони на проектирането**

- Дефинирани са на общодостъпен език
- Независими са от езика за програмиране

4.2.1.3. **Класификация**

- Създидателни - използват се за създаване на обектите (Singleton, Factory, Builder)
- Структурни - използват се за улесняване на интеграцията на класове или обекти
- Поведенчески - представят методи за комуникация между обектите (Observer)

4.2.2. Програмна генерация

4.2.2.1. **Същност**

- Програмните генератори използват вече дефинирани шаблони и алгоритми

- Те са вградени в генератора и може да се параметризират.
- Програмата се генерира автоматично
- Програмната генерация е възможна, когато може да се дефинира съответствие между понятията от приложната област и програмния код
- Използва се специфичен (за проблемната област) език
- Ефективен метод за разработка
- Приложимостта му е в ограничен набор от приложни области
- От гледна точка на крайния потребител е по-лесно в сравнение със стандартните компонентни подходи
-